Q4-2

This is a good question because it relates many concepts such as conversion, elementary rate laws in relation to an overall rate law, and the relationship between concentration and conversion.

i-clicker Question: Ch4

1) If the following liquid-phase reaction is elementary and reversible, what is the rate law in terms of conversion when equimolar amounts of A and B are fed to the reactor?

$$A \leftrightarrow B$$

A) $-r_A = kC_{A0}[1-X]$

B) $-r_A = kC_{A0}[1-X]^2$

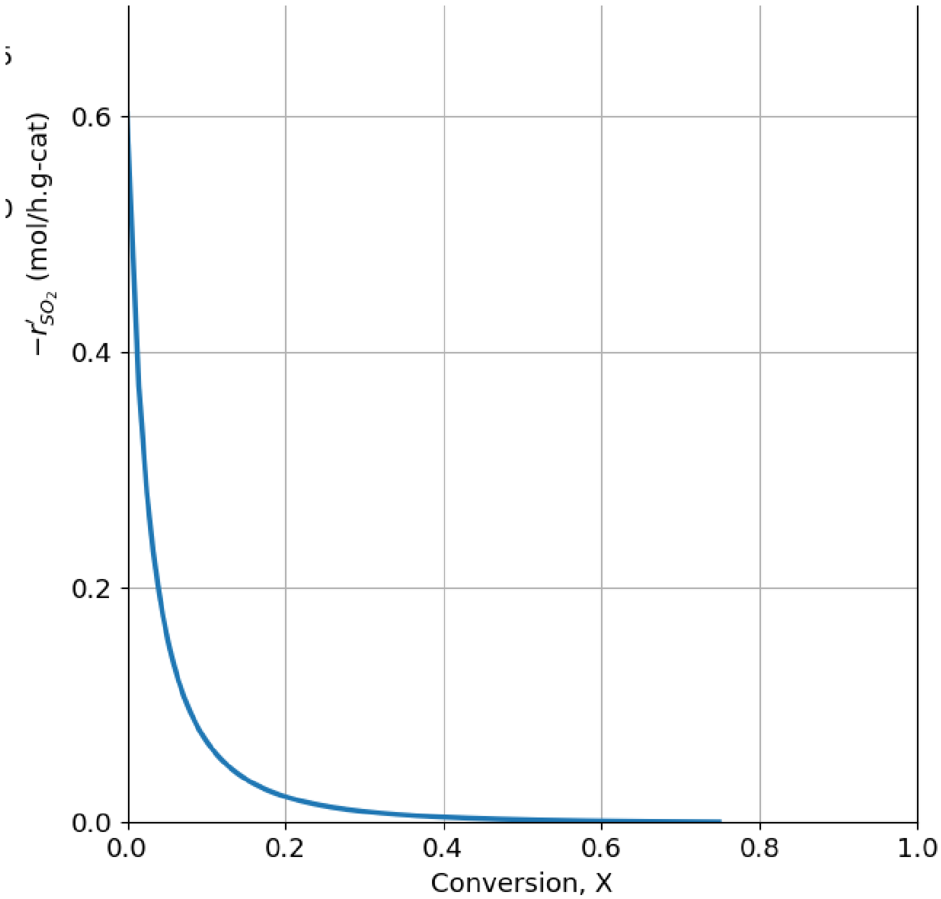
C) $-r_A = kC_{A0}\left([1-X] - \frac{1+X}{K_C}\right)$

D) $-r_A = kC_{A0}\left(1 - X\left[1 + \frac{1}{K_C}\right]\right)$

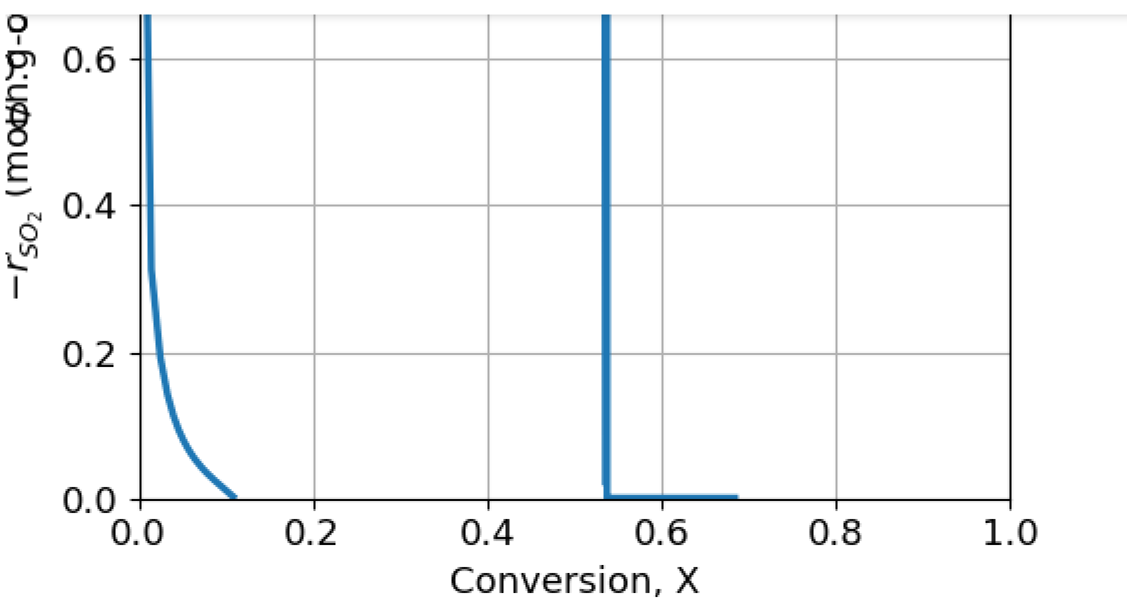
A B C D

P4-1

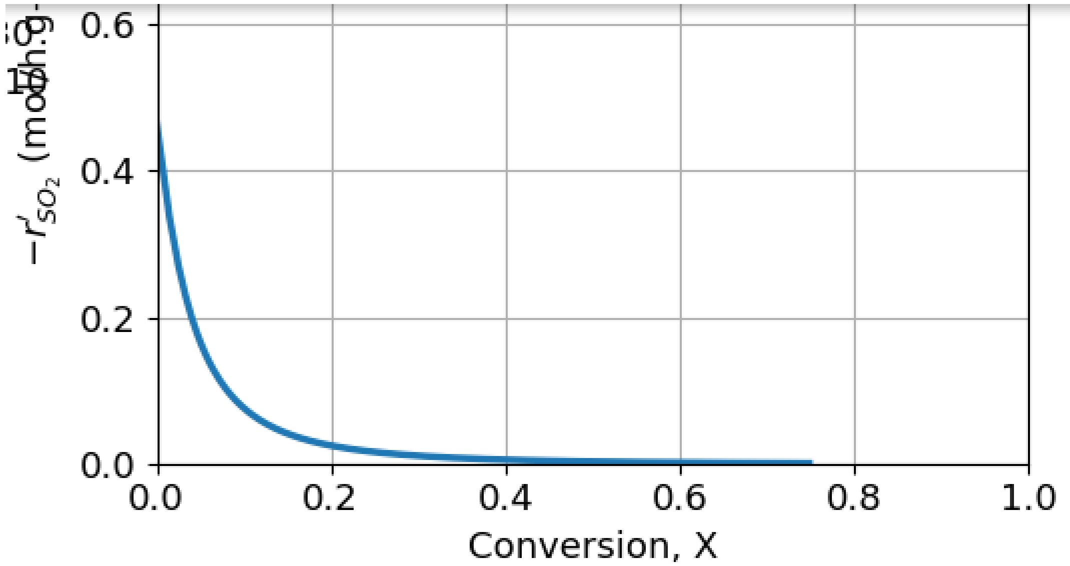
PART 1



Θb initially is at 0.591 which is the first graph shown.



Before hitting zero at Θ = 0.031 the graph has what would most likely be two hump due to the intermediate reaction having a slower rate.



Now when Θ =1.1, double from the original value the rate will be at 0.4 and with increasing it from 1.1 to 2.0 it will only be slightly below 0.4. This is possibly due to the reaction reaching a natural saturation rate.

2.

k directly affects the reaction rate—if k is halved, the rate is halved.

K_{O_2} also impacts the rate, but more strongly than k, because when K_{O_2} increases the rate decreases, and when it decreases the rate increases.

P_{SO_2} affects the rate by increasing the numerator linearly.

K_{SO_3} and K_P have no effect on the rate under these conditions.

3.

Changing Θ which represents the oxygen to SO2 ratio created a graph with two humps showing that there was a slow intermediate reaction with lower values, and with a higher values the rate increase but eventually slowed down. When adjusting, Ko2 was adjusted to be higher and the rate decreased dramatically. Then with k and P_SO2 the rate changed linearly. Overall, oxygen adsorption strength had the greatest effect on reaction rate, showing that the best performance requires comparing oxygen availability against catalyst site inhibition.

PART 4

A

Using formula/code provided by book, and previous hw code, use basic design equation formulas for fluidized bed reactor.

```
#Explicit Equations
thetaB=0.54
epsilon=-0.14
k=9.7
KO2=38.5
KSO3=42.5
KP=930
PSO20=4.1
def myFunction(X, W, thetaB, epsilon, k, KO2, KSO3, KP, PSO20):
    Fa0= 3
    PSO2=PSO20*(1-X)/(1+epsilon*X)
    PSO3=PSO20*X/(1+epsilon* X)
    PO2=PSO20* (thetaB-X/2)/(1+epsilon* X)
    ra= -k*(PSO2* np.sqrt(PO2)-PSO3/KP)/(1+np.sqrt(PO2* KO2)+PSO3 *KSO3)**2
    return -ra/Fa0

#Range of W and intial value of X
W = np.linspace(0, 1000, 10000)
X0 = np.array([0])

def invrate(X, thetaB, epsilon, k, KO2, KSO3, KP, PSO20):
    FA0 = 3

    PSO2 = PSO20*(1 - X)/(1 + epsilon*X)
    PSO3 = PSO20*X/(1 + epsilon*X)
    PO2  = PSO20*(thetaB - X/2)/(1 + epsilon*X)

    ra = -k*(PSO2*np.sqrt(PO2) - PSO3/KP) / \
        (1 + np.sqrt(PO2*KO2) + PSO3*KSO3)**2

    return FA0 / (-ra)
```

Collapse Output

```
# Conversion at inlet and outlet
Xin = 0.0
Xout = 0.4


# Fdesign equation:
# W = (FA0 / -rA)|Xout * (Xout - Xin)
W = invrate(Xout, thetaB, epsilon, k, KO2, KSO3, KP, PSO20 ) * (Xout - Xin)

print("FBR weight =", W, "g")
```

```
FBR weight = 265.2444209261442 g
```

PART 5

```python
#Explicit Equations
thetaB=0.54
epsilon=-0.14
k=9.7
KO2=38.5
KSO3=42.5
KP=930
PSO20=4.1
def myFunction(X, W, thetaB, epsilon, k, KO2, KSO3, KP, PSO20):
    Fa0= 1000
    PSO2=PSO20*(1-X)/(1+epsilon*X)
    PSO3=PSO20*X/(1+epsilon* X)
    PO2=PSO20* (thetaB-X/2)/(1+epsilon* X)
    ra= -k*(PSO2* np.sqrt(PO2)-PSO3/KP)/(1+np.sqrt(PO2* KO2)+PSO3 *KSO3)**2
    return -ra/Fa0

#Range of W and intial value of X
W = np.linspace(0, 1000, 10000)
X0 = np.array([0])

def invrate(X, thetaB, epsilon, k, KO2, KSO3, KP, PSO20):
    FA0 = 1000

    PSO2 = PSO20*(1 - X)/(1 + epsilon*X)
    PSO3 = PSO20*X/(1 + epsilon*X)
    PO2  = PSO20*(thetaB - X/2)/(1 + epsilon*X)

    ra = -k*(PSO2*np.sqrt(PO2) - PSO3/KP) / \
         (1 + np.sqrt(PO2*KO2) + PSO3*KSO3)**2

    return FA0 / (-ra)
```

```python
# Conversion at inlet and outlet
Xin = 0.0
Xout = 0.3

# Fdesign equation:
# W = (FA0 / -rA)|Xout * (Xout - Xin)
W = invrate(Xout, thetaB, epsilon, k, KO2, KSO3, KP, PSO20 ) * (Xout - Xin)

print("FBR weight =", W, "g")
```

```
FBR weight = 32316.888221895177 g
```

```python
def equilibrium(X):
    PSO2 = PSO20*(1 - X)/(1 + epsilon*X)
    PSO3 = PSO20*X/(1 + epsilon*X)
    PO2  = PSO20*(thetaB - X/2)/(1 + epsilon*X)

    return PSO2*np.sqrt(PO2) - PSO3/KP

X_eq = fsolve(equilibrium, 0.7)[0]
print("X equil =", X_eq)

# Conversion at inlet and outlet
Xin = 0.0
Xout = 0.3*X_eq

# Fdesign equation:
# W = (FA0 / -rA)|Xout * (Xout - Xin)
W = invrate(Xout, thetaB, epsilon, k, KO2, KSO3, KP, PSO20 ) * (Xout - Xin)

print("FBR weight =", W, "g")
```

```
X equil = 0.9975795016452569
FBR weight = 32059.24152769153 g
```

B

Using formula/code provided by book, and previous hw code, use basic design equation formulas for PFR

PART 1

```python
#Explicit Equations
Ca0 = 0.072
epsilon= 1
kc= 0.1
ka = 0.5
def myFunction(X, V, Ca0, epsilon, kc, ka):

    Fa0 = 0.359
    ra = -ka*( (Ca0*(1-X)/(1+epsilon*X))
        - ((4*Ca0**2*X**2)/(kc*(1+epsilon*X)**2)))
    return (-ra)/Fa0

#Range of V and intial value of X
V = np.linspace(0, 1000, 10000)
X0 = np.array([0])

def invrate(X, Ca0, epsilon, kc, ka):

    Fa0 = 0.359
    ra = -ka*( (Ca0*(1-X)/(1+epsilon*X))
        - ((4*Ca0**2*X**2)/(kc*(1+epsilon*X)**2)))

    return Fa0/ (-ra)
```

```python
# Conversion limits
Xin = 0.0
Xout = 0.4

# PBR design equation:
# V = ∫(FA0 / -rA) dX from Xin to Xout
V = quad( invrate, Xin, Xout, args=(Ca0, epsilon, kc, ka) )[0]

print("PFR volume =", V, "dm3")
```

```
PFR volume = 8.251193335167725 dm3
```

PART 2

Remove Fao from code because problem gives Cao

```python
def invrate(X, Ca0, epsilon, kc, ka):

    Fa0 = 0.339
    ra = -ka*( (Ca0*(1-X)/(1+epsilon*X))
          - ((4*Ca0**2*X**2)/(kc*(1+epsilon*X)**2)))

    return 1/ (-ra)
```

```python
Ca0 = 0.05
Xout = 0.40
Xin = 0
def batch_time(Xout):
    return Ca0* quad( invrate, Xin, Xout, args=(Ca0, epsilon, kc, ka) )[0]

t = batch_time(Xout)
print(f"Time to reach X = {Xout:.2f}: {t:.3f} min")
```

```
Time to reach X = 0.40: 1.483 min
```

PART 3,4,5,6

3. Kc values between 0.2 and 0.3 and Cao between 0.25 and 0.32 causes Xef to be the farthest away at Xeb. When using the sliders between these values the ratio between Xef and Xeb is around 0.85 to 0.90.

4. When C is 5 and K is 0 the Xef and Xeb are perfectly aligned in a 1 to 1 ratio.

4. As the equilibrium constant increases, both Xeb and Xef increase, but their ratio decreases. As the initial total concentration increases, both equilibrium conversions decrease, but their ratio increases.
5. To solve for PFR volume Simpson's rule must be used because otherwise the answer would be for CSTR. Using Concentration times the integral of dx/-ra I solved for time. K causes both Xeb and Xef to increase and as C increases both Xeb and Xef decreases.

P4-3

A to 1/2B

1/2B to A

Ra = -kfCa+kbCb^1/2

Ra= -kf(Ca-Cb^1/2/Kc)

Flow reactors use Fao